PRAKTIKUM

PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 6

React Dasar



Oleh:

Arif Stand Pramudya 103122400001

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

SOAL PRAKTIKUM REACT DASAR

Aplikasi Manajemen Peserta Praktikum

A. Tugas Program

Buat **satu aplikasi React** untuk mengelola peserta praktikum. Aplikasi harus dapat menambah, mengubah, menghapus, mencari, dan menyaring peserta, serta mengubah status kehadiran dan menampilkan ringkasan data secara langsung.

Seluruh fitur merupakan satu kesatuan program. Gunakan JSX, functional component, props, useState, event handler, serta render list dengan map() dan key. Jawaban berupa proyek yang dapat dijalankan di browser.

B. Data Awal

Ketika aplikasi pertama kali dibuka, tampilkan data berikut. Setiap peserta mempunyai ID unik dan stabil, nama, NIM, kelas, serta status kehadiran.

ID	Nama	NIM	Kelas	Kehadiran
m-1	Ani	10121001	TI-2A	Hadir
m-2	Budi	10121002	TI-2B	Belum hadir
m-3	Citra	10121003	TI-2A	Belum hadir

Gunakan string untuk ID dan NIM. Simpan status kehadiran dalam bentuk boolean: true untuk hadir dan false untuk belum hadir. Peserta baru selalu dimulai dengan status belum hadir.

C. Susunan Tampilan

1. Judul aplikasi dan ringkasan: **Total peserta**, **Hadir**, **Belum hadir**, serta **Hasil ditampilkan**.
2. Form peserta, kolom pencarian, dan filter kelas.
3. Daftar peserta berisi nama, NIM, kelas, status, dan tombol tindakan untuk setiap peserta.

Pada kondisi awal: total peserta 3, hadir 1, belum hadir 2, dan hasil ditampilkan 3. Tampilan boleh berbentuk kartu atau tabel.

KETENTUAN FITUR APLIKASI

D. Form Tambah dan Ubah Peserta

1. Sediakan input **Nama mahasiswa**, input **NIM**, dan pilihan **Kelas**. Pilihan kelas terdiri atas Pilih kelas, TI-2A, dan TI-2B. Pilih kelas merupakan nilai awal dan tidak dianggap sebagai kelas yang valid.
2. Semua input harus terkontrol melalui state. Rapikan spasi di awal dan akhir nama serta NIM. Nama wajib diisi dan memiliki sedikitnya 3 karakter setelah dirapikan; NIM wajib terdiri atas tepat 8 digit dan tetap disimpan sebagai teks.
3. Tolak NIM yang sudah digunakan peserta lain. Saat mengedit, NIM milik peserta yang sedang diedit boleh tetap digunakan. Tampilkan pesan validasi yang jelas; data dan isi form tetap tersedia ketika validasi gagal.
4. Dalam mode tambah, tombol utama bertuliskan **Tambah peserta**. Data valid ditambahkan pada akhir array dengan ID baru, status belum hadir, lalu form dikosongkan dan pilihan kelas kembali ke nilai awal.
5. Tombol **Edit** pada peserta mengisi form dengan data peserta itu. Tombol utama berubah menjadi **Simpan perubahan** dan tombol **Batal** muncul. Penyimpanan memperbarui peserta yang sama tanpa mengganti ID, urutan, atau status kehadirannya.
6. Setelah perubahan berhasil atau Batal ditekan, form kembali ke mode tambah. Batal tidak mengubah data peserta. Tangani onSubmit agar tombol Enter dapat mengirim form tanpa memuat ulang halaman.

E. Hapus dan Kehadiran

1. Tombol **Hapus** menghapus hanya peserta yang dipilih berdasarkan ID. Jika peserta itu sedang diedit, batalkan mode edit dan kosongkan form.
2. Sediakan tombol **Tandai hadir** untuk peserta yang belum hadir, dan **Batalkan hadir** untuk peserta yang hadir. Tombol hanya mengubah status peserta yang dipilih. Status serta ringkasan harus langsung diperbarui.

F. Pencarian dan Filter

Pencarian mencocokkan sebagian nama atau NIM, mengabaikan huruf besar/kecil serta spasi di awal dan akhir kata pencarian. Filter kelas memiliki pilihan Semua kelas, TI-2A, dan TI-2B. Saat keduanya dipakai, tampilkan hanya peserta yang memenuhi kedua kondisi. Pencarian kosong atau hanya spasi berarti tidak membatasi hasil.

KETENTUAN IMPLEMENTASI DAN PENGUMPULAN

G. Ringkasan dan Kondisi Kosong

Total peserta, Hadir, dan Belum hadir dihitung dari seluruh data, sehingga nilainya tidak berubah hanya karena pencarian atau filter. Hasil ditampilkan dihitung dari daftar yang sedang terlihat. Setelah tambah, ubah, hapus, atau perubahan kehadiran, seluruh angka harus mengikuti data terbaru.

Jika seluruh data kosong, tampilkan **Belum ada peserta.** Jika data masih ada tetapi pencarian/filter tidak menghasilkan kecocokan, tampilkan **Peserta tidak ditemukan.** Pencarian dan filter hanya mengubah tampilan, bukan menghapus data sumber.

H. Komponen yang Wajib Dibuat

Komponen	Tanggung jawab
App	Menyimpan state utama, mengolah data, dan menghubungkan seluruh komponen.
FormPeserta	Menampilkan input dan tombol tambah, simpan, serta batal.
FilterPeserta	Menampilkan input pencarian dan pilihan filter kelas.
DaftarPeserta	Merender hasil pencarian/filter dengan map().
ItemPeserta	Menampilkan satu peserta beserta tombol Edit, Hapus, dan kehadiran.
RingkasanPeserta	Menampilkan empat angka ringkasan dari data yang diterima.

Simpan data utama, nilai form, mode edit, pencarian, filter, dan pesan dalam state di App. Kirim data serta fungsi penanganan event melalui props. Komponen anak tidak boleh mengubah props secara langsung. Gunakan ID stabil sebagai key dan perbarui array/object melalui setter state.

I. Menjalankan dan Mengumpulkan Program

Gunakan React + JavaScript (.jsx) dengan Vite. Dari folder proyek, jalankan perintah berikut secara berurutan; buka alamat yang muncul pada terminal.

```
npm install
npm run dev
```

Data cukup disimpan dalam state selama aplikasi berjalan. Reload mengembalikan data awal. Sebelum dikumpulkan, pastikan **npm run build** berhasil.

Kumpulkan satu ZIP proyek berisi src, index.html, package.json, package-lock.json, konfigurasi Vite, dan README singkat cara menjalankan. Sertakan tangkapan layar kondisi awal, hasil tambah/edit, hasil pencarian, dan kondisi kosong. Keluarkan node_modules dan dist dari ZIP.

SKENARIO PENGUJIAN PROGRAM

Jalankan pengujian secara berurutan. Pada pengujian yang menolak input, pastikan jumlah peserta dan data tersimpan tidak berubah.

No.	Tindakan	Hasil yang diharapkan
1	Buka aplikasi.	Total 3; hadir 1; belum hadir 2; terlihat 3 peserta.
2	Coba kirim form kosong, nama An, NIM 12A, atau kelas belum dipilih. Uji setiap kasus dengan isian lain valid.	Setiap input yang tidak sesuai ditolak dengan pesan yang jelas; total tetap 3.
3	Tambah Dina dengan NIM 10121001 dan kelas TI-2B.	Ditolak karena NIM sudah digunakan Ani; total tetap 3.
4	Tambah Dina dengan NIM 00123001 dan kelas TI-2B.	Total 4; hadir 1; belum hadir 3. NIM tetap 00123001; form kembali kosong.
5	Klik Tandai hadir pada Budi.	Total 4; hadir 2; belum hadir 2. Status Budi menjadi hadir.
6	Dengan filter Semua kelas, cari nama menggunakan teks " bU ".	Hanya Budi terlihat; hasil ditampilkan 1. Ringkasan seluruh data tetap 4, 2, dan 2.
7	Pertahankan pencarian bU, lalu pilih filter TI-2A.	Hasil ditampilkan 0; Peserta tidak ditemukan. tampil; data sumber tetap ada.
8	Kosongkan pencarian dan pilih Semua kelas. Edit Dina menjadi Dina Putri, kelas TI-2A, NIM tetap.	Perubahan diterima; total tetap 4. ID dan status belum hadir milik Dina tetap.
9	Edit lagi Dina Putri, ubah NIM menjadi 10121002, lalu simpan. Setelah ditolak, klik Batal.	NIM ganda ditolak; data sebelumnya tetap. Batal mengosongkan form dan mengaktifkan mode tambah.
10	Klik Edit pada Ani, lalu hapus Ani dari daftar.	Ani terhapus; mode edit dibatalkan. Total 3; hadir 1; belum hadir 2.
11	Hapus seluruh peserta yang tersisa.	Semua ringkasan bernilai 0; Belum ada peserta. tampil.
12	Muat ulang halaman.	Tiga peserta awal tampil kembali; total 3, hadir 1, belum hadir 2.

Program dinyatakan selesai apabila seluruh fitur dapat dicoba melalui antarmuka, skenario di atas berjalan sesuai ketentuan, dan Console browser tidak menampilkan error JavaScript atau peringatan key yang hilang.

JAWABAN PRAKTIKUM REACT DASAR

Aplikasi Manajemen Peserta Praktikum

A. Kode Program

- App.jsx

```jsx
import { useMemo, useState } from "react";
import FormPeserta from "./components/FormPeserta.jsx";
import FilterPeserta from "./components/FilterPeserta.jsx";
import DaftarPeserta from "./components/DaftarPeserta.jsx";
import RingkasanPeserta from "./components/RingkasanPeserta.jsx";

const dataAwal = [
  { id: "m-1", nama: "Ani", nim: "10121001", kelas: "TI-2A", hadir: true },
  { id: "m-2", nama: "Budi", nim: "10121002", kelas: "TI-2B", hadir: false },
  { id: "m-3", nama: "Citra", nim: "10121003", kelas: "TI-2A", hadir: false
},
];

const formKosong = { nama: "", nim: "", kelas: "" };

export default function App() {
  const [peserta, setPeserta] = useState(dataAwal);
  const [formData, setFormData] = useState(formKosong);
  const [editingId, setEditingId] = useState(null);
  const [pencarian, setPencarian] = useState("");
  const [filterKelas, setFilterKelas] = useState("semua");
  const [pesan, setPesan] = useState(null);

  const pesertaTerlihat = useMemo(() => {
    const kata = pencarian.trim().toLocaleLowerCase();

    return peserta.filter((item) => {
      const cocokKata =
        kata === "" ||
        item.nama.toLocaleLowerCase().includes(kata) ||
        item.nim.toLocaleLowerCase().includes(kata);

      const cocokKelas =
        filterKelas === "semua" || item.kelas === filterKelas;

      return cocokKata && cocokKelas;
    });
  }, [peserta, pencarian, filterKelas]);

  const jumlahHadir = peserta.filter((item) => item.hadir).length;
  const ringkasan = {
    total: peserta.length,
    hadir: jumlahHadir,
    belumHadir: peserta.length - jumlahHadir,
    hasil: pesertaTerlihat.length,
```

```
  };

  function resetForm() {
    setFormData(formKosong);
    setEditingId(null);
  }

  function ubahField(field, value) {
    setFormData((sebelumnya) => ({ ...sebelumnya, [field]: value }));
  }

  function mulaiEdit(item) {
    setFormData({ nama: item.nama, nim: item.nim, kelas: item.kelas });
    setEditingId(item.id);
    setPesan({
      tipe: "info",
      teks: `Mode edit aktif untuk peserta ${item.nama}.`,
    });
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function batalEdit() {
    resetForm();
    setPesan({ tipe: "info", teks: "Perubahan dibatalkan. Form siap untuk
menambah peserta." });
  }

  function simpanPeserta(event) {
    event.preventDefault();

    const nama = formData.nama.trim();
    const nim = formData.nim.trim();
    const kelas = formData.kelas;

    setFormData({ nama, nim, kelas });

    if (!nama) {
      setPesan({ tipe: "error", teks: "Nama mahasiswa wajib diisi." });
      return;
    }

    if (nama.length < 3) {
      setPesan({ tipe: "error", teks: "Nama harus memiliki minimal 3 karakter
setelah spasi dirapikan." });
      return;
    }

    if (!/^\d{8}$/.test(nim)) {
      setPesan({ tipe: "error", teks: "NIM harus terdiri dari tepat 8 digit
angka." });
      return;
```

```
    }

    if (!["TI-2A", "TI-2B"].includes(kelas)) {
      setPesan({ tipe: "error", teks: "Silakan pilih kelas TI-2A atau TI-2B."
});
      return;
    }

    const nimSudahDipakai = peserta.some(
      (item) => item.nim === nim && item.id !== editingId
    );

    if (nimSudahDipakai) {
      setPesan({ tipe: "error", teks: `NIM ${nim} sudah digunakan peserta
lain. Gunakan NIM yang berbeda.` });
      return;
    }

    if (editingId) {
      setPeserta((sebelumnya) =>
        sebelumnya.map((item) =>
          item.id === editingId
            ? { ...item, nama, nim, kelas }
            : item
        )
      );
      resetForm();
      setPesan({ tipe: "success", teks: `Data ${nama} berhasil diperbarui.`
});
      return;
    }

    const pesertaBaru = {
      id: `m-${Date.now()}-${Math.random().toString(36).slice(2, 8)}`,
      nama,
      nim,
      kelas,
      hadir: false,
    };

    setPeserta((sebelumnya) => [...sebelumnya, pesertaBaru]);
    resetForm();
    setPesan({ tipe: "success", teks: `${nama} berhasil ditambahkan sebagai
peserta baru.` });
  }

  function hapusPeserta(id) {
    const itemDihapus = peserta.find((item) => item.id === id);
    setPeserta((sebelumnya) => sebelumnya.filter((item) => item.id !== id));

    if (editingId === id) {
```

```
      resetForm();
    }

    setPesan({
      tipe: "success",
      teks: itemDihapus ? `Data ${itemDihapus.nama} berhasil dihapus.` :
"Data peserta berhasil dihapus.",
    });
  }

  function ubahKehadiran(id) {
    const item = peserta.find((pesertaItem) => pesertaItem.id === id);
    setPeserta((sebelumnya) =>
      sebelumnya.map((pesertaItem) =>
        pesertaItem.id === id
          ? { ...pesertaItem, hadir: !pesertaItem.hadir }
          : pesertaItem
      )
    );

    if (item) {
      setPesan({
        tipe: "success",
        teks: `Status kehadiran ${item.nama} diubah menjadi ${item.hadir ?
"belum hadir" : "hadir"}.`,
      });
    }
  }

  const pesanKosong =
    peserta.length === 0
      ? "Belum ada peserta."
      : "Peserta tidak ditemukan.";

  return (
    <main className="app-shell">
      <header className="topbar">
        <a className="brand" href="#" aria-label="Manajemen Peserta
Praktikum">
          <span className="brand-mark">SM</span>
          <span>
            <strong>Student Management<span className="brand-
dot">.</span></strong>
            <small>Praktikum</small>
          </span>
        </a>
        <span className="topbar-tag"><span className="online-dot" />
Praktikum · Modul 06</span>
      </header>

      <section className="page-heading">
```

```
        <div>
          <p className="eyebrow">DASHBOARD PRAKTIKUM</p>
          <h1>Manajemen Peserta</h1>
          <p className="page-description">
            Kelola data mahasiswa, pantau kehadiran, dan temukan peserta
dengan lebih mudah.
          </p>
        </div>
        <div className="heading-date">
          <span className="date-icon">▦</span>
          <span><strong>Data praktikum</strong><small>Perbarui data secara
langsung</small></span>
        </div>
      </section>

      <RingkasanPeserta ringkasan={ringkasan} />

      {pesan && (
        <div className={`alert alert-${pesan.tipe}`} role="status" aria-
live="polite">
          <span className="alert-symbol">
            {pesan.tipe === "error" ? "!" : pesan.tipe === "success" ? "✓" :
"i"}
          </span>
          <span>{pesan.teks}</span>
          <button className="alert-close" type="button" onClick={() =>
setPesan(null)} aria-label="Tutup pesan">×</button>
        </div>
      )}

      <section className="workspace">
        <aside className="form-panel panel">
          <div className="panel-heading">
            <div className="panel-icon form-icon">+</div>
            <div>
              <h2>{editingId ? "Edit peserta" : "Tambah peserta"}</h2>
              <p>{editingId ? "Perbarui informasi peserta yang dipilih." :
"Lengkapi informasi mahasiswa baru."}</p>
            </div>
          </div>
          <FormPeserta
            formData={formData}
            onFieldChange={ubahField}
            onSubmit={simpanPeserta}
            editing={Boolean(editingId)}
            onCancel={batalEdit}
          />
          <div className="form-note">
            <span className="note-icon">i</span>
            <p>Pastikan NIM terdiri dari 8 digit angka dan belum digunakan
peserta lain.</p>
```

```
          </div>
        </aside>

        <section className="list-panel panel">
          <div className="list-heading">
            <div>
              <h2>Daftar peserta</h2>
              <p>Daftar mahasiswa yang terdaftar pada praktikum.</p>
            </div>
            <span className="count-pill">{peserta.length} peserta</span>
          </div>

          <FilterPeserta
            pencarian={pencarian}
            onPencarianChange={setPencarian}
            filterKelas={filterKelas}
            onFilterKelasChange={setFilterKelas}
          />

          <DaftarPeserta
            peserta={pesertaTerlihat}
            totalPeserta={peserta.length}
            pesanKosong={pesanKosong}
            onEdit={mulaiEdit}
            onHapus={hapusPeserta}
            onUbahKehadiran={ubahKehadiran}
          />

          <div className="list-footer">
            <span>Menampilkan <strong>{pesertaTerlihat.length}</strong> dari
<strong>{peserta.length}</strong> peserta</span>
            <span className="footer-status"><span className="online-dot" />
Data diperbarui langsung</span>
          </div>
        </section>
      </section>

      <footer className="page-footer">
        <span>Manajemen Peserta Praktikum</span>
        <span>Arif Stand Pramudya | 103122400001</span>
      </footer>
    </main>
  );
}
```

- components/DaftarPeserta.jsx

```
import ItemPeserta from "./ItemPeserta.jsx";

export default function DaftarPeserta({
  peserta,
  totalPeserta,
  pesanKosong,
```

```
  onEdit,
  onHapus,
  onUbahKehadiran,
}) {
  if (peserta.length === 0) {
    return (
      <div className="empty-state">
        <div className="empty-illustration">
          <span className="empty-paper">▤</span>
          <span className="empty-search">⌕</span>
        </div>
        <h3>{pesanKosong}</h3>
        <p>
          {totalPeserta === 0
            ? "Peserta yang ditambahkan akan muncul di bagian ini."
            : "Coba ubah kata pencarian atau pilih filter kelas yang lain."}
        </p>
      </div>
    );
  }

  return (
    <div className="participant-list">
      <div className="list-column-head">
        <span>MAHASISWA</span>
        <span>KELAS</span>
        <span>KEHADIRAN</span>
        <span className="actions-heading">AKSI</span>
      </div>
      {peserta.map((item) => (
        <ItemPeserta
          key={item.id}
          peserta={item}
          onEdit={() => onEdit(item)}
          onHapus={() => onHapus(item.id)}
          onUbahKehadiran={() => onUbahKehadiran(item.id)}
        />
      ))}
    </div>
  );
}
```

- components/FilterPeserta.jsx

```
export default function FilterPeserta({
  pencarian,
  onPencarianChange,
  filterKelas,
  onFilterKelasChange,
}) {
  return (
    <div className="filters">
      <label className="search-box" htmlFor="pencarian">
```

```
        <span className="search-icon" aria-hidden="true">⌕</span>
        <input
          id="pencarian"
          type="search"
          placeholder="Cari nama atau NIM..."
          value={pencarian}
          onChange={(event) => onPencarianChange(event.target.value)}
        />
        {pencarian && (
          <button
            type="button"
            className="clear-search"
            onClick={() => onPencarianChange("")}
            aria-label="Hapus pencarian"
          >
            ×
          </button>
        )}
      </label>

      <label className="filter-select-wrap" htmlFor="filter-kelas">
        <span className="filter-label">Kelas</span>
        <select
          id="filter-kelas"
          value={filterKelas}
          onChange={(event) => onFilterKelasChange(event.target.value)}
        >
          <option value="semua">Semua kelas</option>
          <option value="TI-2A">TI-2A</option>
          <option value="TI-2B">TI-2B</option>
        </select>
        <span className="select-chevron" aria-hidden="true">⌄</span>
      </label>
    </div>
  );
}
```

- components/FormPeserta.jsx

```
export default function FormPeserta({
  formData,
  onFieldChange,
  onSubmit,
  editing,
  onCancel,
}) {
  return (
    <form className="participant-form" onSubmit={onSubmit} noValidate>
      <div className="field-group">
        <label htmlFor="nama">Nama mahasiswa <span
className="required">*</span></label>
        <input
          id="nama"
```

```
          name="nama"
          type="text"
          placeholder="Contoh: Dina Putri"
          value={formData.nama}
          onChange={(event) => onFieldChange("nama", event.target.value)}
          autoComplete="name"
        />
        <small>Minimal 3 karakter.</small>
      </div>

      <div className="field-group">
        <label htmlFor="nim">NIM <span className="required">*</span></label>
        <input
          id="nim"
          name="nim"
          type="text"
          inputMode="numeric"
          maxLength={8}
          placeholder="Contoh: 00123001"
          value={formData.nim}
          onChange={(event) => onFieldChange("nim", event.target.value)}
          autoComplete="off"
        />
        <small>Masukkan tepat 8 digit angka.</small>
      </div>

      <div className="field-group">
        <label htmlFor="kelas">Kelas <span
className="required">*</span></label>
        <select
          id="kelas"
          name="kelas"
          value={formData.kelas}
          onChange={(event) => onFieldChange("kelas", event.target.value)}
        >
          <option value="">Pilih kelas</option>
          <option value="TI-2A">TI-2A</option>
          <option value="TI-2B">TI-2B</option>
        </select>
      </div>

      <div className="form-actions">
        <button className="button button-primary" type="submit">
          <span>{editing ? "✓" : "+"}</span>
          {editing ? "Simpan perubahan" : "Tambah peserta"}
        </button>
        {editing && (
          <button className="button button-secondary" type="button"
onClick={onCancel}>
            Batal
          </button>
```

```
        )}
      </div>
    </form>
  );
}
```

- components/ItemPeserta.jsx

```
export default function ItemPeserta({
  peserta,
  onEdit,
  onHapus,
  onUbahKehadiran,
}) {
  const inisial = peserta.nama
    .trim()
    .split(/\s+/)
    .slice(0, 2)
    .map((bagian) => bagian[0]?.toUpperCase() ?? "")
    .join("");

  return (
    <article className="participant-row">
      <div className="student-cell">
        <div className={`avatar ${peserta.hadir ? "avatar-present" : "avatar-
absent"}`}>
          {inisial}
        </div>
        <div className="student-details">
          <strong>{peserta.nama}</strong>
          <span>NIM {peserta.nim}</span>
        </div>
      </div>

      <div className="class-cell">
        <span className="class-badge">{peserta.kelas}</span>
      </div>

      <div className="attendance-cell">
        <span className={`status-badge ${peserta.hadir ? "status-present" :
"status-absent"}`}>
          <span className="status-dot" />
          {peserta.hadir ? "Hadir" : "Belum hadir"}
        </span>
      </div>

      <div className="action-cell">
        <button className="icon-button edit-button" type="button"
onClick={onEdit} title="Edit peserta" aria-label={`Edit ${peserta.nama}`}>
          <span aria-hidden="true">✎</span>
        </button>
        <button
          className={`icon-button attendance-button ${peserta.hadir ? "undo-
```

```
button" : "check-button"}`}
          type="button"
          onClick={onUbahKehadiran}
          title={peserta.hadir ? "Batalkan hadir" : "Tandai hadir"}
          aria-label={peserta.hadir ? `Batalkan hadir ${peserta.nama}` :
`Tandai hadir ${peserta.nama}`}
        >
          <span aria-hidden="true">{peserta.hadir ? "↶" : "✓"}</span>
        </button>
        <button className="icon-button delete-button" type="button"
onClick={onHapus} title="Hapus peserta" aria-label={`Hapus ${peserta.nama}`}>
          <span aria-hidden="true">×</span>
        </button>
      </div>
    </article>
  );
}
```

- components/RingkasanPeserta.jsx

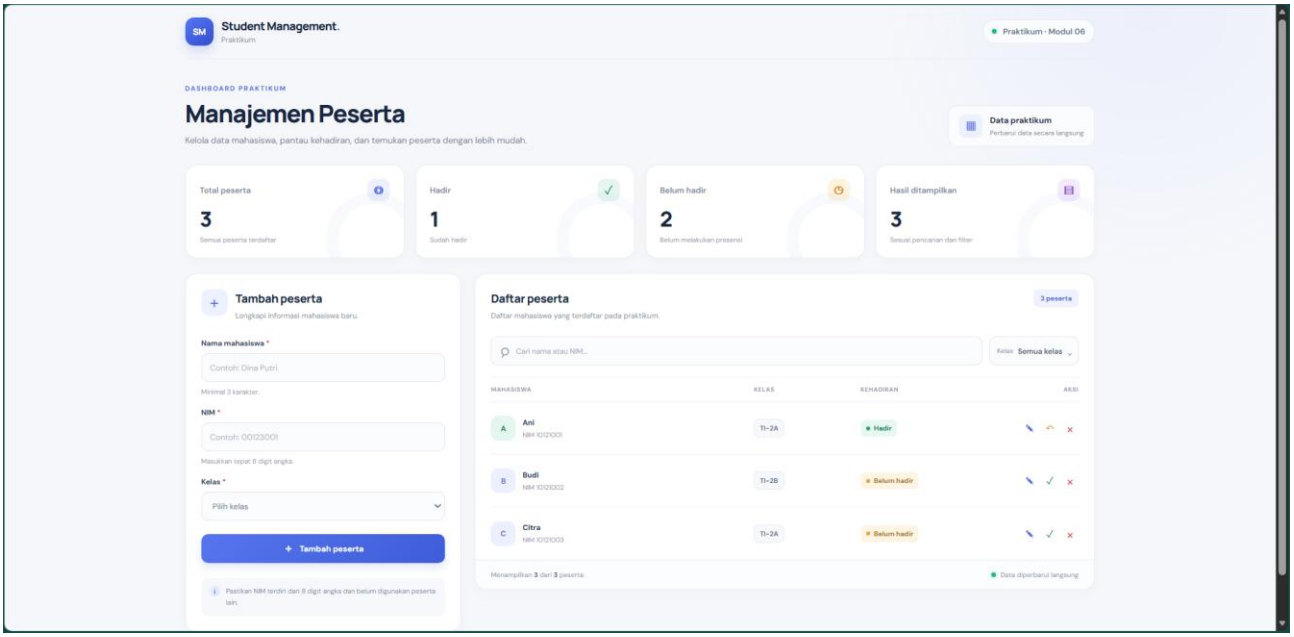
```
const kartuRingkasan = [
  {
    key: "total",
    label: "Total peserta",
    description: "Semua peserta terdaftar",
    icon: "◎",
    className: "stat-total",
  },
  {
    key: "hadir",
    label: "Hadir",
    description: "Sudah hadir",
    icon: "✓",
    className: "stat-present",
  },
  {
    key: "belumHadir",
    label: "Belum hadir",
    description: "Belum melakukan presensi",
    icon: "◷",
    className: "stat-absent",
  },
  {
    key: "hasil",
    label: "Hasil ditampilkan",
    description: "Sesuai pencarian dan filter",
    icon: "▤",
    className: "stat-results",
  },
];

export default function RingkasanPeserta({ ringkasan }) {
  return (
```

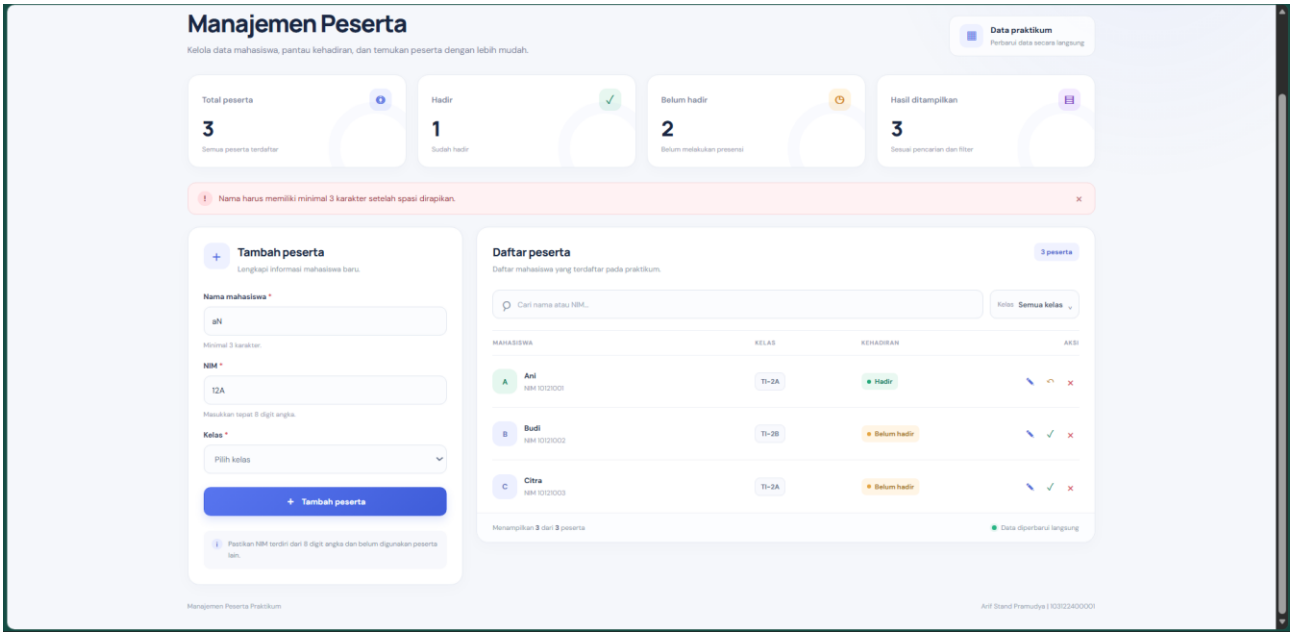
```
    <section className="stats-grid" aria-label="Ringkasan peserta">
      {kartuRingkasan.map((kartu) => (
        <article className={`stat-card ${kartu.className}`} key={kartu.key}>
          <div className="stat-top">
            <span className="stat-label">{kartu.label}</span>
            <span className="stat-icon" aria-
hidden="true">{kartu.icon}</span>
          </div>
          <div className="stat-value">{ringkasan[kartu.key]}</div>
          <div className="stat-description">{kartu.description}</div>
        </article>
      ))}
    </section>
  );
}
```

B. Screenshot Output

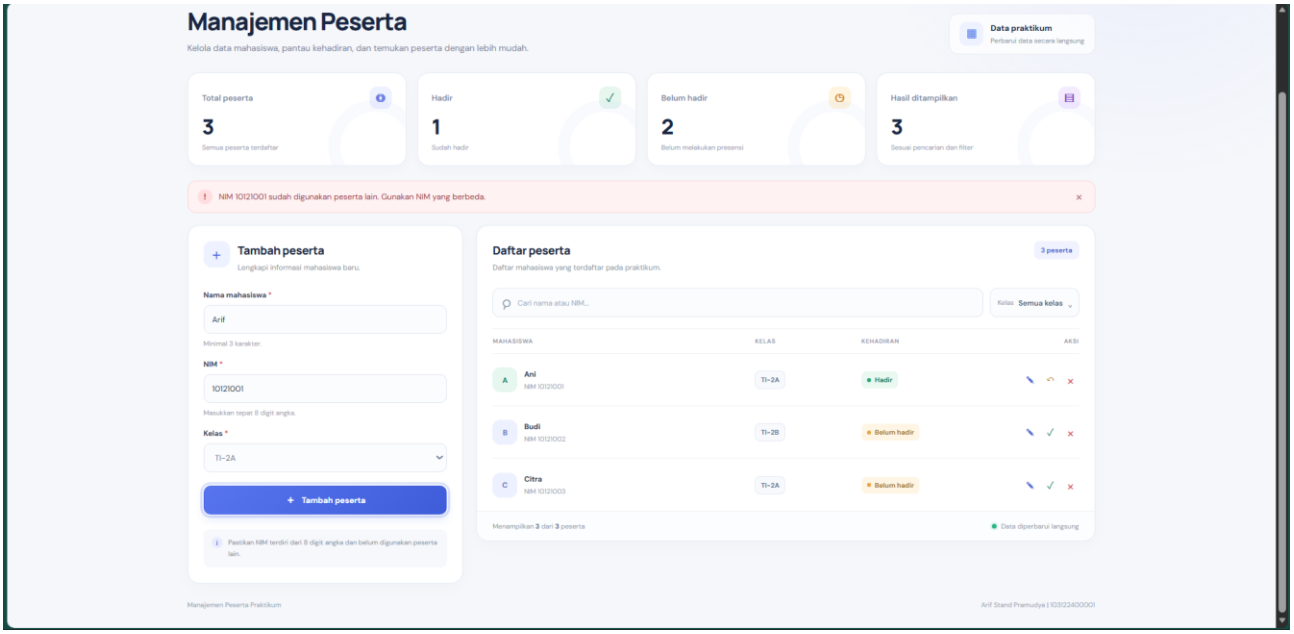
- Buka aplikasi



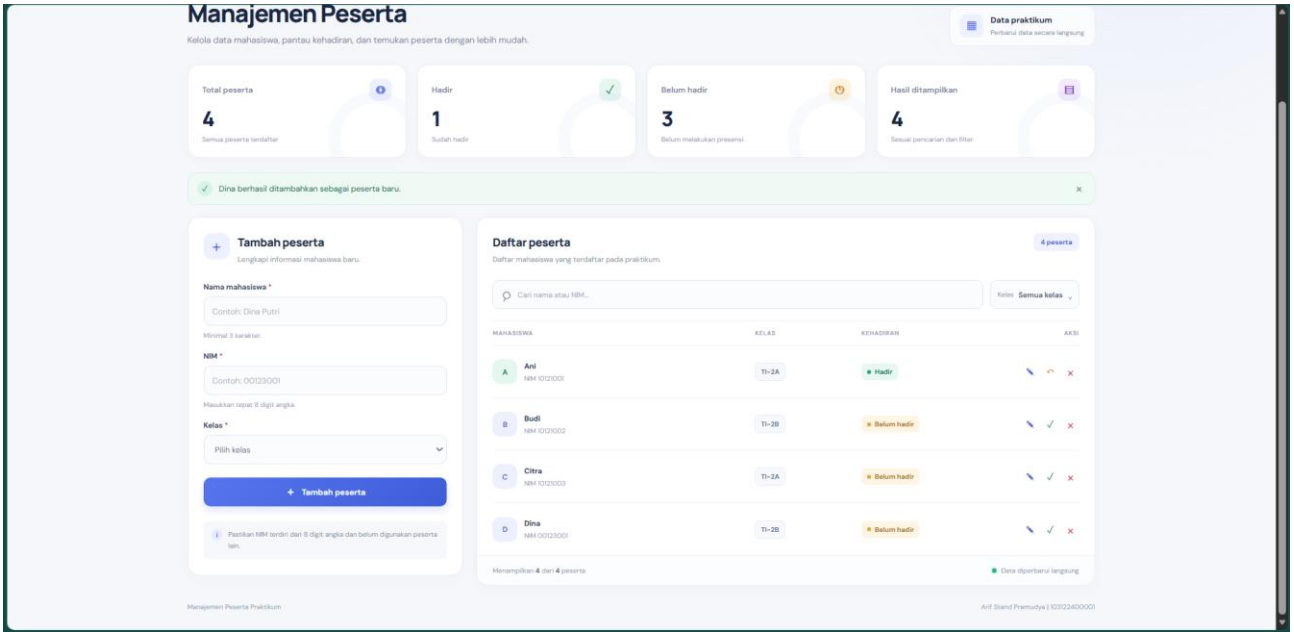
- Uji validasi form



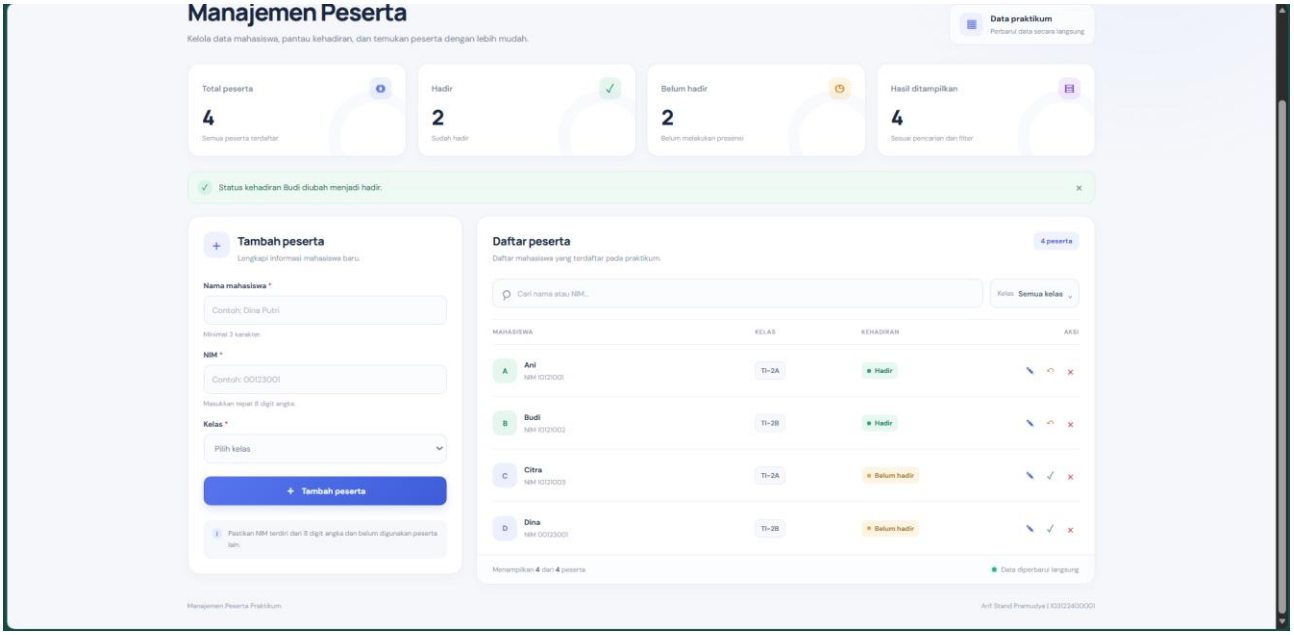
- Uji NIM duplikat



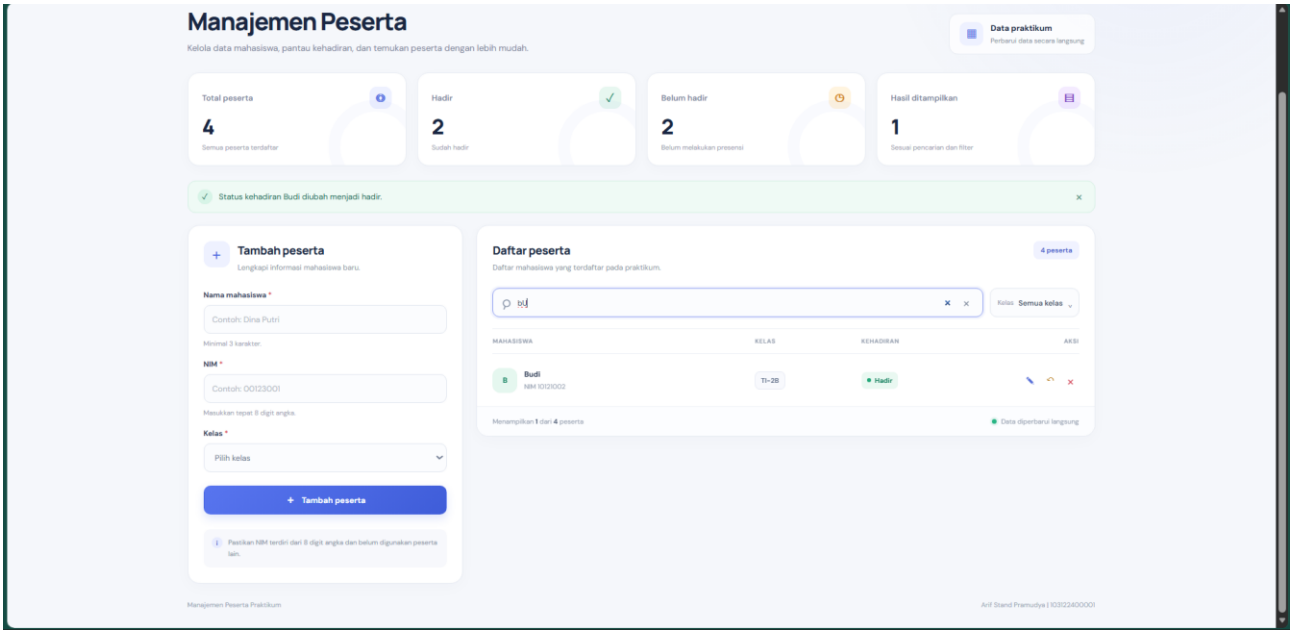
- Tambah peserta baru



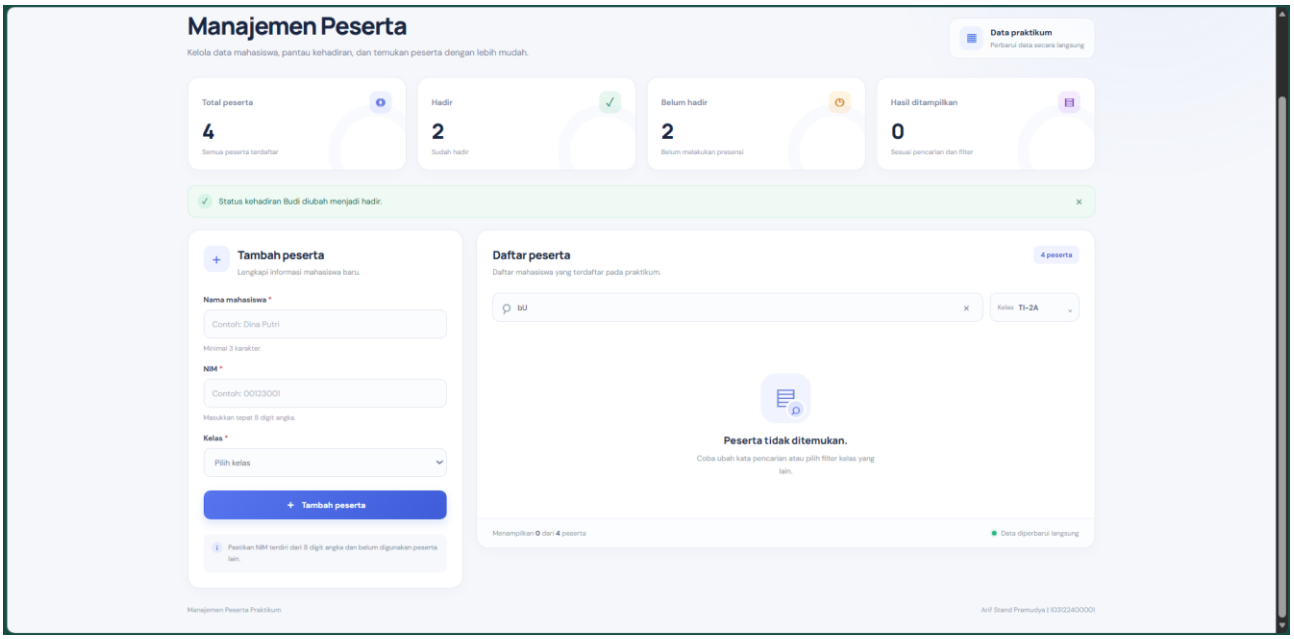
- Ubah kehadiran



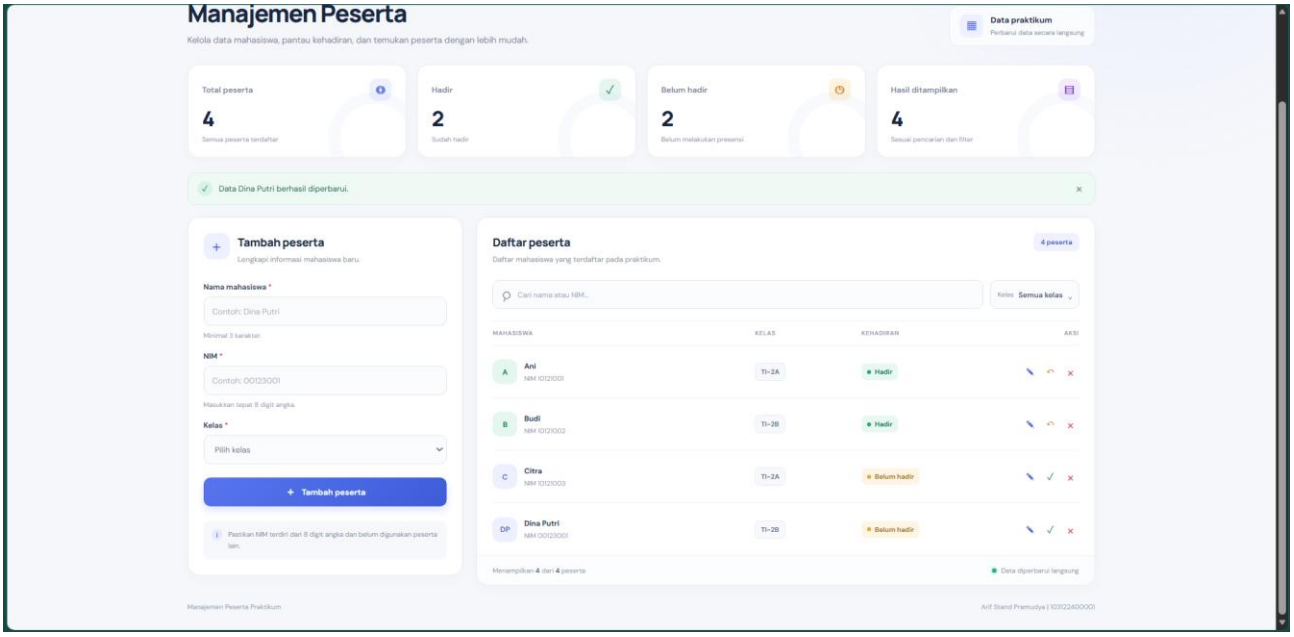
- Uji pencarian



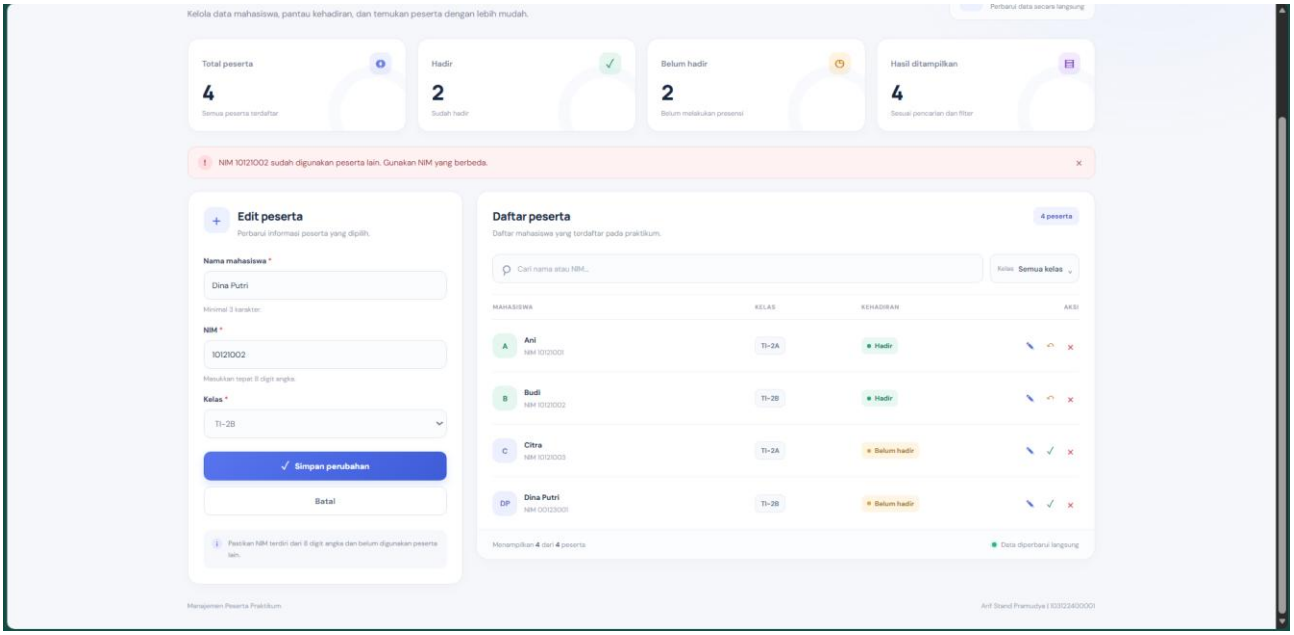
- Gabungkan pencarian dan filter



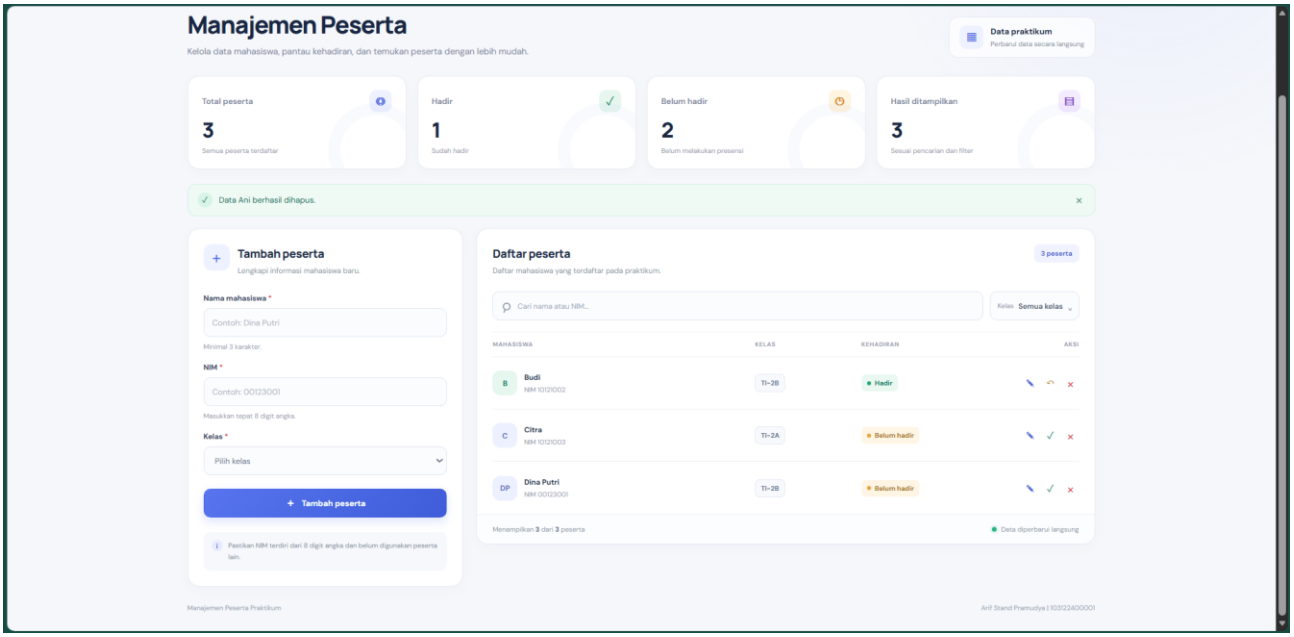
- Edit peserta



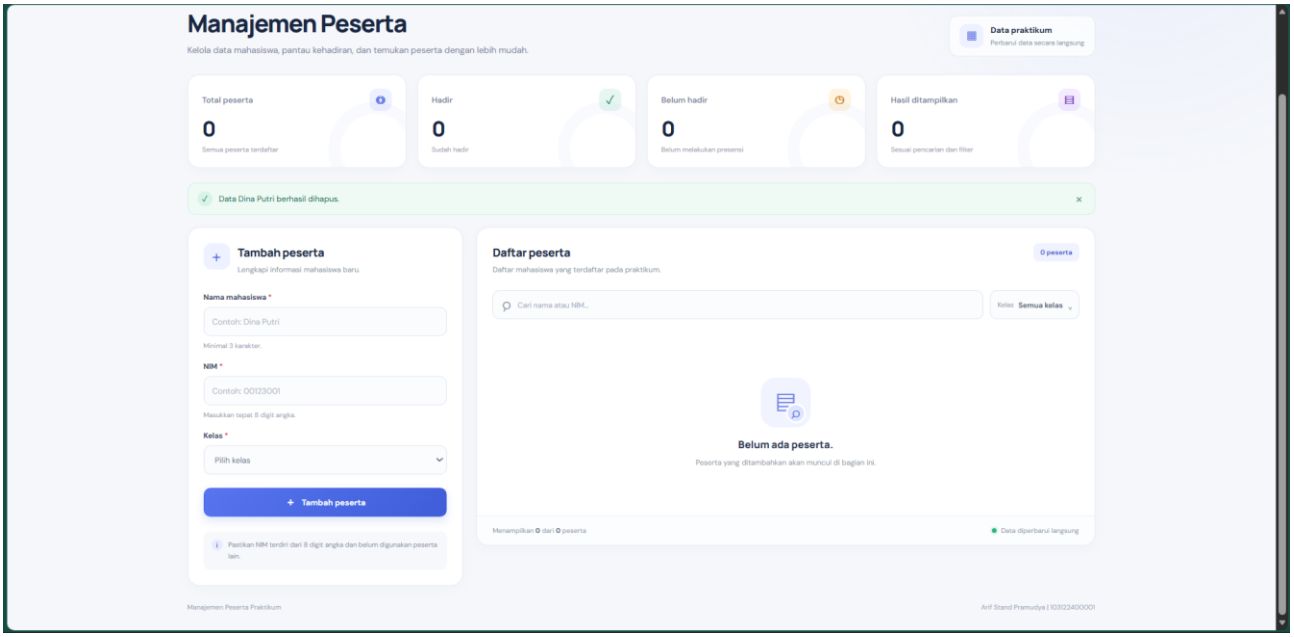
- Uji validasi saat edit



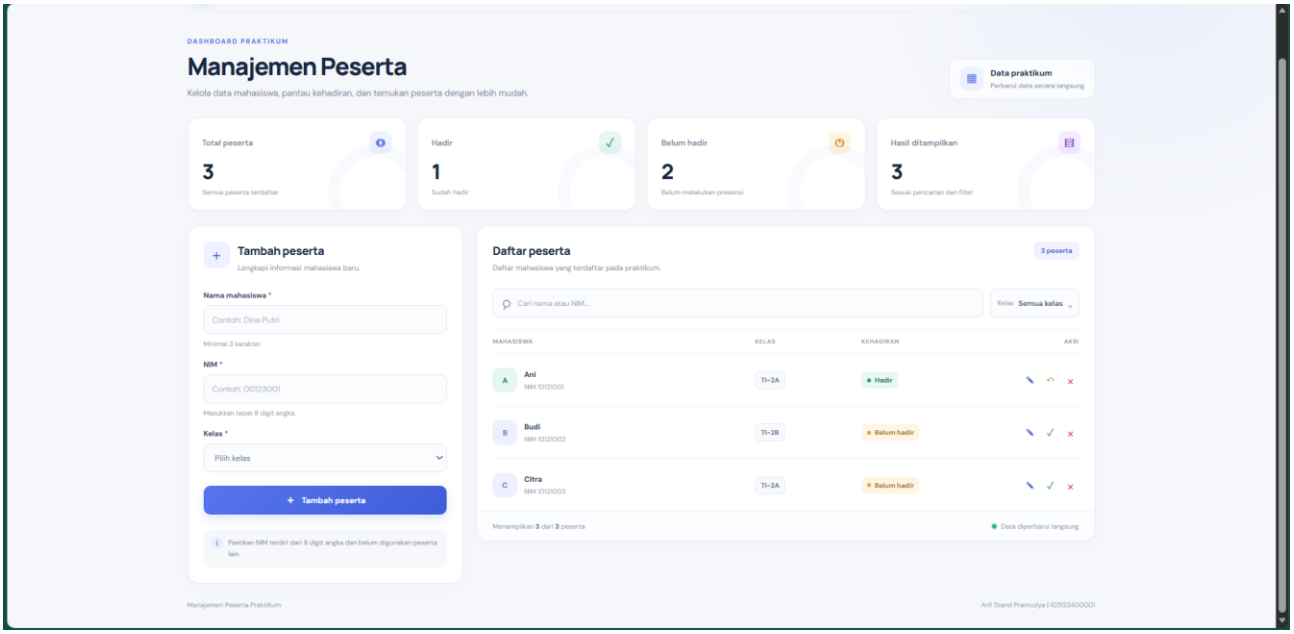
- Hapus saat mode edit



- Hapus seluruh peserta



- Muat ulang halaman



C. Deskripsi Program

Pada modul 6 react dasar ini saya membuat program manajemen peserta praktikum berbasis React yang digunakan untuk mengelola data mahasiswa praktikum. Program ini memiliki fitur tambah, edit, hapus, pencarian, filter kelas, dan pengelolaan status kehadiran. Aplikasi menggunakan useState untuk menyimpan dan memperbarui data secara langsung, serta map() untuk menampilkan daftar peserta. Ringkasan jumlah peserta, hadir, dan belum hadir diperbarui otomatis sesuai perubahan data.